

AGEX Deployment Architecture

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Deployment Architecture의 구조, 배포 단위, Control Plane과 Data Plane 배치, 환경 분리, Container, Kubernetes 기반 운영 가능 구조, Runtime Worker, Storage, Network, Region, Availability, Scaling, Upgrade, Rollback, Backup, Disaster Recovery, Observability 및 Infrastructure Abstraction 기준을 정의한다.

AGEX는 단일 Web Application이 아니라 Agent, Workflow, Knowledge, Memory, Plugin, Model Gateway, Runtime 및 Marketplace가 상호 연동되는 AI Operating System이므로 모든 구성요소를 하나의 단일 서버나 단일 프로세스에 결합하는 구조를 기본으로 하지 않는다.

Deployment Architecture는 다음 요구를 동시에 만족해야 한다.

- SaaS 배포
- Private Cloud 배포
- On-Premises 배포
- Hybrid 배포
- Multi-Region 배포
- Shared Tenant 배포
- Dedicated Tenant 배포
- GPU Runtime 배포
- Local/Development 환경
- 확장 가능한 Provider 교체

본 문서는 AGEX의 실제 인프라 구축 및 운영 설계의 상위 기준으로 사용한다.

1.2 적용 범위

본 문서는 다음 영역을 포함한다.

- Deployment Model
- Environment Architecture

- Control Plane Deployment
- Data Plane Deployment
- Runtime Deployment
- Worker Pool Architecture
- Container Architecture
- Orchestration
- Service Deployment
- Network Architecture
- Storage Architecture
- Database Architecture
- Queue and Event Infrastructure
- Cache
- Search
- Vector Infrastructure
- Model Gateway
- GPU Infrastructure
- Secrets
- Configuration
- Multi-Region
- High Availability
- Scaling
- Upgrade
- Rollback
- Backup
- Disaster Recovery

- Infrastructure as Code
- Observability
- Deployment Security
- Capacity Planning

1.3 Deployment Architecture의 공식 정의

AGEX Deployment Architecture는 AGEX의 모든 Control, Execution, Data 및 Extension Component를 독립적으로 배포·확장·복구할 수 있도록 구성하는 Cloud-Native, Provider-Agnostic 실행 인프라 구조이다.

2. Deployment Architecture 목표

AGEX Deployment Architecture는 다음 목표를 충족해야 한다.

2.1 독립적 확장

API, Runtime Worker, Model Gateway, Knowledge Retrieval 및 기타 부하 특성이 다른 Component를 독립적으로 확장할 수 있어야 한다.

2.2 장애 격리

하나의 Worker, Plugin, Model Provider 또는 Service 장애가 전체 AGEX 장애로 확산되지 않아야 한다.

2.3 배포 독립성

핵심 Service를 필요한 경우 독립적으로 배포할 수 있어야 한다.

2.4 Provider 독립성

특정 Cloud Vendor의 독점 기능 없이는 운영할 수 없는 구조를 Core Architecture로 채택하지 않는다.

2.5 환경 일관성

Development, Testing, Staging, Production에서 동일한 Artifact와 Deployment Contract를 사용해야 한다.

2.6 확장 가능한 격리

Shared Infrastructure에서 Dedicated Tenant Infrastructure까지 동일한 AGEX Contract로

확장할 수 있어야 한다.

2.7 복구 가능성

장애, 잘못된 Release 또는 Infrastructure Failure 발생 시 Rollback과 Recovery가 가능해야 한다.

2.8 운영 자동화

Infrastructure, Deployment, Scaling 및 Configuration을 가능한 한 선언적으로 관리해야 한다.

2.9 보안

Service Identity, Secret, Network 및 Runtime Boundary를 Deployment Layer에서도 강제해야 한다.

2.10 관측 가능성

모든 배포 단위의 Health, Metric, Log 및 Trace를 중앙에서 확인할 수 있어야 한다.

3. Deployment 핵심 원칙

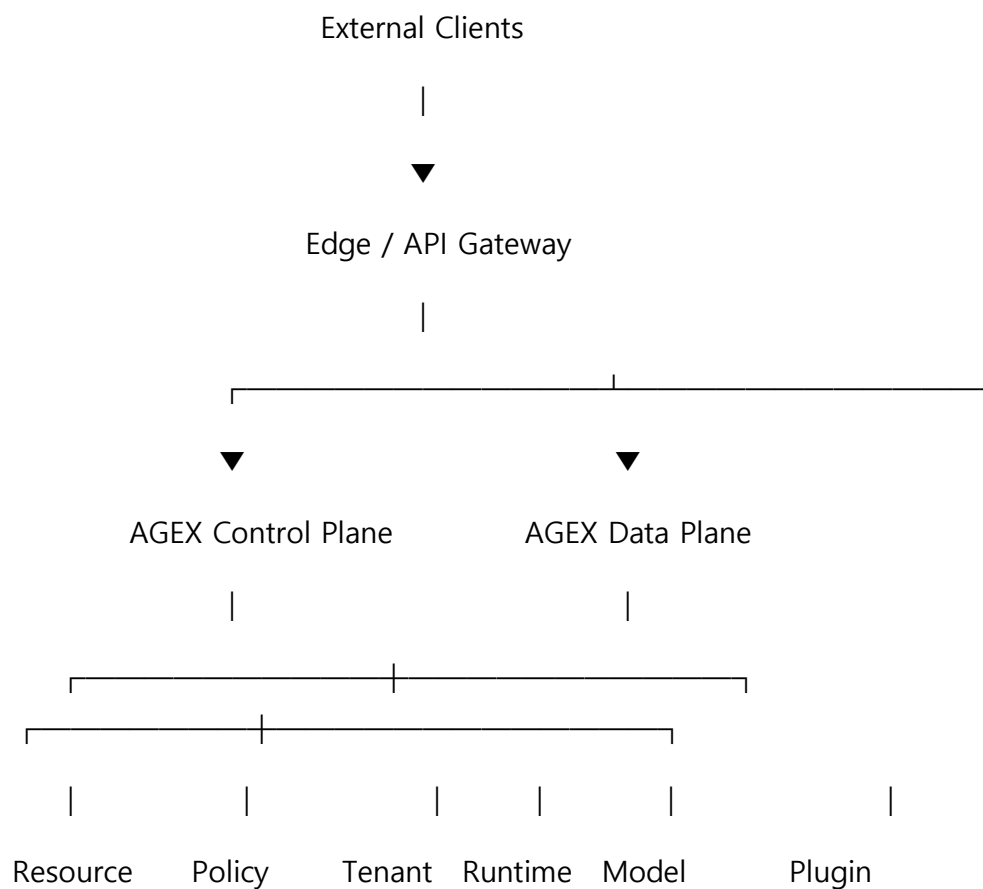
AGEX는 다음 Deployment 원칙을 따른다.

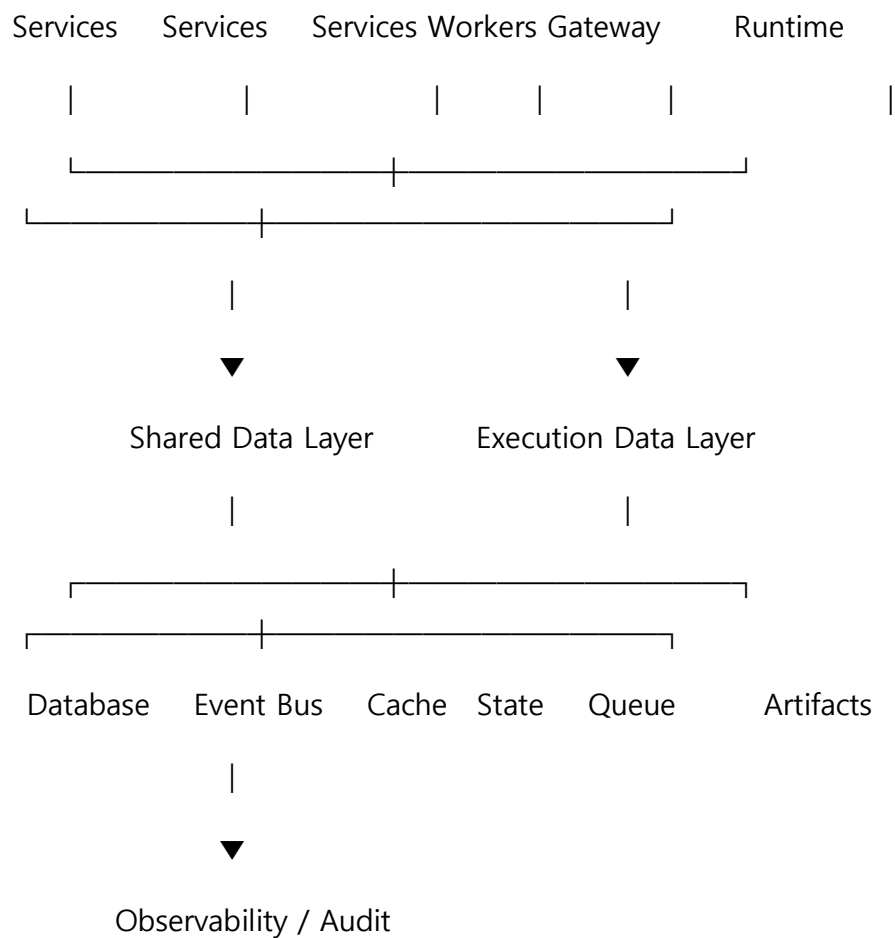
- Cloud Native
- Container First
- Infrastructure as Code
- Immutable Artifact
- Stateless Service Where Possible
- Externalized State
- Horizontal Scaling
- Independent Worker Pools
- Control Plane / Data Plane Separation
- Zero Trust Network
- Rolling Deployment

- Canary Ready
- Graceful Shutdown
- Graceful Drain
- Automated Health Check
- Multi-Zone Ready
- Multi-Region Ready
- Provider Abstraction
- No Production Snowflake Server
- Observable by Default

4. Deployment 전체 구조

AGEX의 권장 논리 Deployment 구조는 다음과 같다.





5. Control Plane과 Data Plane

5.1 Control Plane

Control Plane은 AGEX Resource, Policy, Tenant, Configuration, Registry 및 Governance를 관리한다.

주요 Component:

- API Gateway
- Resource Service
- Registry Service
- Tenant Service
- Identity Integration
- Policy Service

- Configuration Service
- Schema Service
- Marketplace Control Service
- Usage Control
- Deployment Controller

5.2 Data Plane

Data Plane은 실제 AI와 Workflow 실행을 처리한다.

주요 Component:

- Execution Manager
- Scheduler
- Dispatcher
- Runtime Worker
- Agent Worker
- Workflow Worker
- Plugin Runtime
- Model Gateway
- Sandbox
- Knowledge Retrieval Runtime
- Memory Runtime

5.3 분리 목적

- 장애 격리
- 독립 Scaling
- 보안 Boundary
- Dedicated Tenant 지원
- GPU Resource 분리

- Region별 Data Plane 배치

5.4 Control Plane 장애

Control Plane 일부가 일시적으로 장애가 발생해도 이미 검증된 Execution Snapshot을 가진 Data Plane 작업은 정책 범위 내에서 계속 실행할 수 있도록 설계할 수 있다.

단, 새로운 권한 결정이나 긴급 Security Block이 필요한 상황에서는 안전을 우선한다.

6. Deployment Model

AGEX는 최소 다음 Deployment Model을 지원할 수 있는 구조를 가져야 한다.

6.1 Public SaaS

AGEX Platform 운영자가 전체 Infrastructure를 운영한다.

6.2 Dedicated SaaS

특정 Tenant 전용 Data Plane 또는 Infrastructure를 제공한다.

6.3 Private Cloud

Tenant의 Cloud Account 또는 전용 Cloud 환경에 배포한다.

6.4 On-Premises

Tenant가 소유한 Data Center 또는 Private Infrastructure에 배포한다.

6.5 Hybrid

Control Plane과 일부 Data Plane을 서로 다른 환경에 배치한다.

예:

AGEX Control Plane

|



Private Connector

|



Tenant Private Runtime



Private Data / Local Model

6.6 Edge 또는 Restricted Runtime

필요한 경우 제한된 Runtime을 원격 또는 폐쇄 환경에 배치할 수 있다.

7. Environment Architecture

최소 다음 Environment를 분리한다.

- Local
- Development
- Integration
- Testing
- Staging
- Production

7.1 환경 격리

각 환경은 최소한 다음을 분리한다.

- Credentials
- Database
- Queue
- Secrets
- Storage
- Runtime
- API Endpoint

7.2 Production Data

Production 데이터를 Development 환경에 자동 복제하지 않는다.

7.3 Artifact

동일한 Build Artifact를 환경별로 재빌드하지 않고 Promotion하는 방식을 권장한다.

Build Once

→ Test

→ Stage

→ Approve

→ Production

8. Local Development Deployment

Local 환경은 전체 Production Cluster를 요구해서는 안 된다.

8.1 Local 구성

다음 Component를 경량 구성으로 제공할 수 있다.

- API
- Core Service
- Runtime
- Local Queue
- Database
- Cache
- Mock Model
- Mock Plugin
- Local Object Store

8.2 Container Compose

초기 개발과 Local Test에서는 Container Compose와 같은 경량 Orchestration을 사용할 수 있다.

8.3 Production 차이

Local Runtime이 Production 보안 및 고가용성을 동일하게 보장한다고 간주하지 않는다.

9. Container Architecture

9.1 Container 단위

다음 Component는 독립 Container 또는 배포 단위가 될 수 있다.

- API Gateway
- Core API
- Policy Service
- Runtime Manager
- Scheduler
- Dispatcher
- Worker
- Plugin Runtime
- Model Gateway
- Knowledge Service
- Memory Service
- Marketplace Service

9.2 Immutable Image

Production Container Image는 실행 중 변경하지 않는다.

9.3 Configuration

Configuration은 Image 외부에서 주입한다.

9.4 Secret

Secret은 Container Image에 포함하지 않는다.

9.5 Root

가능한 경우 Non-Root User로 실행한다.

10. Container Image Standard

Production Image는 다음 정보를 추적할 수 있어야 한다.

- Image Name
- Image Version
- Build ID
- Source Revision
- Base Image
- Dependency Lock
- SBOM
- Checksum
- Signature
- Build Provenance

10.1 Latest Tag 금지

Production Deployment가 latest와 같이 불변성을 보장하지 않는 Tag에만 의존하지 않는다.

10.2 Digest Pinning

고신뢰 환경에서는 Image Digest를 고정할 수 있다.

11. Container Orchestration

AGEX는 Container Orchestration Layer를 통해 다음 기능을 제공받을 수 있어야 한다.

- Scheduling
- Service Discovery
- Health Check

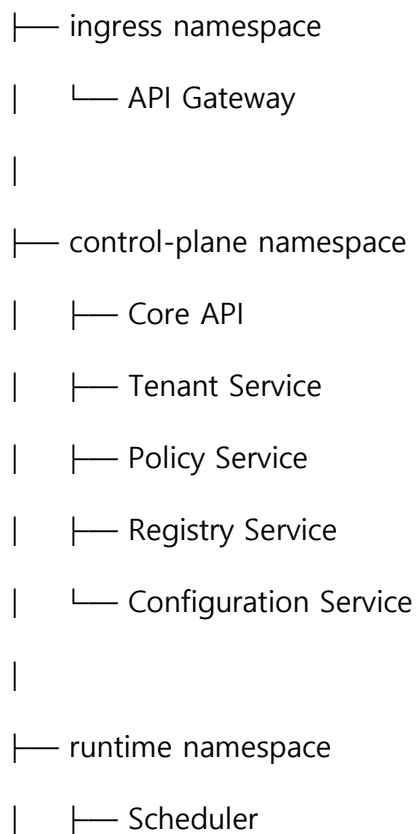
- Restart
- Scaling
- Rolling Update
- Secret Injection
- Config Injection
- Resource Limit
- Network Policy

Kubernetes는 주요 구현 후보이지만 AGEX Domain이 Kubernetes API에 직접 종속되지 않아야 한다.

12. Kubernetes Reference Architecture

Kubernetes 기반 배포 시 예시 구조는 다음과 같다.

Cluster



```
|   ├── Dispatcher
|   ├── Agent Workers
|   ├── Workflow Workers
|   └── Plugin Workers
|
|── model namespace
|   ├── Model Gateway
|   └── Private Model Runtime
|
|── data namespace
|   ├── State Store
|   ├── Queue
|   └── Cache
|
|── observability namespace
|   ├── Metrics
|   ├── Logging
|   └── Tracing
```

실제 Namespace 구조는 배포 규모에 맞게 변경할 수 있다.

13. Service Deployment Strategy

AGEX는 초기에는 일부 기능을 Modular Monolith로 배포할 수 있다.

중요한 원칙은 논리 Domain Boundary가 유지되는 것이다.

13.1 초기 Deployment

예:

agex-core

agex-runtime-manager

agex-worker

agex-model-gateway

agex-plugin-runtime

13.2 성장 후

부하와 조직 요구에 따라 다음을 독립 Service로 분리할 수 있다.

- Tenant Service
- Policy Service
- Registry Service
- Audit Service
- Usage Service
- Knowledge Service
- Memory Service

13.3 Microservice 강제 금지

초기부터 모든 Domain을 독립 Microservice로 분리하여 운영 복잡도를 불필요하게 높이지 않는다.

14. Stateless Service

가능한 Service는 Stateless하게 운영한다.

예:

- API Gateway
- Public API
- Query Service
- Dispatcher

상태는 외부 State Store에 저장한다.

14.1 Local Memory 금지

중요한 Workflow 또는 Execution State를 특정 Pod Memory에만 유지하지 않는다.

15. Stateful Component

다음 Component는 Stateful Infrastructure가 필요하다.

- Primary Database
- Execution State Store
- Event Log
- Object Storage
- Search Index
- Vector Store
- Cache
- Queue

15.1 관리 방식

가능한 경우 Managed Service 또는 검증된 Stateful Operator를 사용할 수 있다.

15.2 Provider 추상화

Domain Logic은 해당 Managed Product의 독점 API에 직접 결합하지 않는다.

16. Database Architecture

16.1 기본 역할

RDBMS는 다음과 같은 Transactional State의 주요 후보이다.

- Resource Metadata
- Tenant
- Policy Metadata

- Execution Metadata
- Workflow State
- Registry
- Configuration

16.2 요구사항

- Transaction
- Strong Consistency
- Replication
- Backup
- Point-in-Time Recovery
- Encryption
- Failover

16.3 Database 선택

구체적인 Product는 Deployment Profile에서 결정한다.

AGEX Core Architecture는 특정 RDBMS Vendor에 종속되지 않는다.

17. Database High Availability

Production Database는 가능한 경우 다음을 지원해야 한다.

- Primary / Replica
- Automatic Failover
- Multi-Zone Placement
- Backup
- PITR
- Health Monitoring

17.1 Application 연결

Failover 이후 Application이 자동으로 새로운 Primary와 연결될 수 있어야 한다.

18. Database Migration

Schema Migration은 Application Deployment와 별도로 관리 가능한 절차를 가져야 한다.

18.1 기본 전략

Expand

→ Deploy Compatible Application

→ Data Migration

→ Verify

→ Contract

18.2 Breaking Migration

즉시 Column 삭제와 같이 이전 Version을 깨뜨리는 변경을 피한다.

18.3 Rollback

Application Rollback과 Database Rollback 가능성을 함께 검토해야 한다.

19. Execution State Store

Execution State는 높은 내구성을 가져야 한다.

19.1 저장 대상

- Execution State
- Task State
- Workflow State
- Checkpoint Metadata
- Lease
- Retry
- Timeout

- Side Effect Ledger

19.2 Source of Truth

Queue가 아니라 Transactional State Store가 Execution 상태의 Source of Truth이다.

20. Object Storage

Object Storage는 다음에 사용할 수 있다.

- Artifact
- Large Input
- Large Output
- Knowledge Source
- Package
- Export
- Backup Artifact

20.1 Metadata

Object 자체와 함께 Tenant, Classification, Checksum 및 Retention을 관리한다.

20.2 Version

필요한 Resource는 Object Versioning을 사용할 수 있다.

20.3 Lifecycle

Cold Storage 또는 Expiration 정책을 적용할 수 있다.

21. Cache Architecture

Cache는 다음 영역에서 사용할 수 있다.

- Resource Read
- Policy Metadata
- Model Metadata

- Session
- Rate Limit
- Retrieval Result

21.1 Source of Truth 금지

Cache는 Source of Truth가 아니다.

21.2 Tenant Isolation

Cache Key에 Tenant Scope를 포함한다.

21.3 장애

Cache 전체 장애가 Core Data 손실로 이어지지 않아야 한다.

22. Queue Architecture

Queue는 비동기 실행의 핵심 Infrastructure이다.

22.1 Queue 유형

- Execution Queue
- Agent Queue
- Workflow Queue
- Plugin Queue
- Model Queue
- Retry Queue
- Scheduled Queue
- Compensation Queue
- DLQ

22.2 전달 보장

기본적으로 At-Least-Once Delivery를 전제로 한다.

따라서 Consumer는 Idempotent해야 한다.

22.3 Queue Scaling

Worker 수와 Queue Depth에 따라 독립적으로 Scaling할 수 있어야 한다.

23. Event Infrastructure

Event Bus는 Domain 및 Integration Event를 전달한다.

23.1 요구사항

- Partition
- Ordering Where Required
- Replay
- Consumer Group
- Retention
- DLQ
- Schema Version

23.2 Queue와 Event Bus 구분

Queue는 Task Dispatch 목적이 강하다.

Event Bus는 상태 변화 전달 목적이 강하다.

두 역할을 논리적으로 구분한다.

24. Search Infrastructure

Search Infrastructure는 다음에 사용될 수 있다.

- Resource Search
- Audit Search
- Marketplace Search
- Knowledge Keyword Search

24.1 Source of Truth

Search Index는 Derived Data이다.

24.2 Rebuild

원본 Resource에서 재생성 가능해야 한다.

24.3 Tenant Scope

Search Index에서도 Tenant Filter를 강제한다.

25. Vector Infrastructure

Vector Store는 다음 영역에서 사용할 수 있다.

- Knowledge
- Memory
- Semantic Search

25.1 Provider Abstraction

Vector Store는 Adapter Interface를 사용한다.

25.2 Partition

Tenant, Collection, Region 또는 Classification 기준으로 Partition할 수 있다.

25.3 Reindex

Embedding Model 변경 시 새 Index Version을 구축할 수 있어야 한다.

26. Graph Infrastructure

Knowledge Graph 또는 Resource Dependency Graph를 위해 Graph Store를 선택적으로 사용할 수 있다.

26.1 필수 Component 아님

AGEX Core가 특정 Graph Database 없이는 실행되지 않는 구조를 만들지 않는다.

27. Model Gateway Deployment

Model Gateway는 Model Provider와 AGEX Runtime 사이의 공통 Gateway로 독립 배포한다.

주요 역할:

- Model Routing
- Provider Adapter
- Credential Broker Integration
- Rate Limit
- Cost Tracking
- Retry
- Fallback
- Streaming
- Policy Enforcement

27.1 독립 Scaling

Model 요청량에 따라 API Core와 독립적으로 확장할 수 있어야 한다.

28. Model Provider Adapter Deployment

Provider Adapter는 다음 방식으로 배포할 수 있다.

- Model Gateway 내부 Module
- 독립 Adapter Service
- Remote Provider Connector

28.1 선택 기준

- Provider 규모
- Security
- Throughput
- 독립 Release 필요성

- Protocol 차이

29. Private Model Runtime

Local 또는 Self-Hosted Model을 사용할 경우 별도 AI Runtime Layer를 둘 수 있다.

구조 예:

AGEX Model Gateway

|



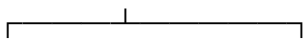
Private Model Adapter

|



Inference Gateway

|



GPU Pool A GPU Pool B

29.1 Model Server

구현 시 vLLM, TGI 또는 기타 Runtime을 사용할 수 있지만 AGEX Model Contract가 특정 제품에 종속되지 않아야 한다.

30. GPU Worker Architecture

GPU Resource는 일반 Runtime Worker와 분리할 수 있다.

30.1 GPU Pool 유형

- Inference
- Embedding

- Re-ranking
- Fine-Tuning Extension
- Multimodal

30.2 Scheduling

GPU 요구사항을 Task Metadata에 명시한다.

30.3 Resource 요청

- GPU Type
- GPU Count
- Memory
- Model
- Runtime

30.4 일반 Task

GPU가 필요 없는 Task를 GPU Worker에서 실행하여 Resource를 낭비하지 않는다.

31. GPU Autoscaling

GPU Worker는 다음 Signal로 확장할 수 있다.

- Queue Depth
- Pending Tasks
- GPU Utilization
- Model Request Rate

31.1 Scale-to-Zero

비실시간 Worker는 비용 절감을 위해 Scale-to-Zero를 지원할 수 있다.

31.2 Cold Start

Scale-to-Zero의 Model Loading 시간을 Latency Policy에 반영해야 한다.

32. Agent Worker Deployment

Agent Worker는 Agent Loop 실행을 담당한다.

32.1 Worker 특성

- Stateless where possible
- Short-Lived Task Context
- Lease Based
- Checkpoint Integration
- Model Gateway Access
- Plugin Gateway Access

32.2 Scale

Agent Queue와 실행 수에 따라 확장한다.

33. Workflow Worker Deployment

Workflow Worker는 State Machine과 Step Coordination을 담당한다.

33.1 장기 대기

Wait 상태 동안 Worker를 점유하지 않는다.

33.2 Scheduler

대기 종료 시 다시 Queue에 투입한다.

34. Plugin Runtime Deployment

Plugin Runtime은 Core와 분리한다.

34.1 Trust 기반 Pool

예:

Certified Plugin Pool

Verified Plugin Pool

Restricted Sandbox Pool

34.2 Plugin별 독립 Container

위험 수준에 따라 Action 단위 또는 Plugin 단위의 독립 Container를 사용할 수 있다.

35. Sandbox Infrastructure

고위험 코드 실행은 별도 Sandbox Infrastructure에서 수행한다.

지원 가능한 격리:

- Container Sandbox
- Hardened Container
- MicroVM
- Restricted Process

35.1 Network

Default Deny Egress를 사용한다.

35.2 Cleanup

Execution 종료 후 환경을 폐기한다.

36. Knowledge Processing Worker

Knowledge Ingestion은 Retrieval API와 분리된 Worker Pool에서 처리할 수 있다.

36.1 Task

- Parsing
- Chunking
- Metadata Extraction
- Embedding
- Indexing

36.2 이유

대량 Ingestion이 실시간 Retrieval Latency에 영향을 주지 않도록 한다.

37. Memory Processing Worker

Memory Consolidation, Summarization 및 Expiration은 비동기 Worker에서 처리할 수 있다.

37.1 실시간 Retrieval

Memory Retrieval은 별도의 Latency 최적화 Service에서 처리할 수 있다.

38. Marketplace Validation Worker

Marketplace Package에 대한 다음 작업은 별도 격리 Worker에서 수행한다.

- Package Extraction
- Malware Scan
- Dependency Scan
- Contract Test
- Runtime Test

검증되지 않은 Package를 Core Service Process에서 직접 실행하지 않는다.

39. API Gateway Deployment

API Gateway는 Edge Layer에 배치한다.

39.1 역할

- TLS
- Routing
- Authentication Integration
- Rate Limit
- Request Size

- Threat Filtering
- API Version Routing

39.2 Scale

Stateless Horizontal Scaling을 적용한다.

39.3 Multi-Zone

Production에서 복수 Failure Domain에 배치한다.

40. Load Balancing

Load Balancer는 Stateless Service 트래픽을 분배한다.

40.1 Session Affinity

가능한 한 Session Affinity에 의존하지 않는다.

40.2 Streaming

Streaming Connection을 지원할 수 있는 설정이 필요하다.

40.3 Health

Unhealthy Instance에는 신규 Traffic을 전달하지 않는다.

41. Service Discovery

내부 Service Endpoint를 Hard Coding하지 않는다.

Service Discovery 또는 Runtime Configuration을 사용한다.

41.1 DNS

Container Orchestration 환경에서는 DNS 기반 Discovery를 사용할 수 있다.

41.2 External Service

외부 Provider Endpoint는 Registry 및 Configuration으로 관리한다.

42. Network Architecture

권장 Network Zone:

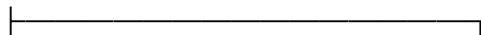
Internet



Edge Zone



API / Control Zone



Runtime Zone

Integration Zone



Data Zone

External Providers

42.1 Data Zone

Database와 Secret Infrastructure는 외부에서 직접 접근할 수 없어야 한다.

43. Network Policy

Service 간 허용 Network를 명시적으로 정의한다.

예:

- API → Core
- Core → Database
- Runtime → Model Gateway
- Runtime → Plugin Gateway

- Plugin Runtime → Allowlisted External Endpoint

43.1 All-to-All 금지

Cluster 내부 모든 Service가 모든 Service에 접근 가능한 기본 Network를 지양한다.

44. Egress Architecture

외부 통신은 가능한 경우 Egress Gateway 또는 Proxy를 통해 통제한다.

44.1 장점

- Allowlist
- Logging
- Rate Limit
- Data Policy
- IP Control

44.2 Plugin

Plugin Runtime의 직접 Internet 접근을 제한한다.

45. Ingress Architecture

외부 공개 Endpoint를 최소화한다.

Public 후보:

- API Gateway
- Webhook Gateway
- Console Frontend

Private 유지 후보:

- Database
- Queue
- Internal Services

- Runtime Manager
 - Model Worker
 - Secret Store
-

46. Secret Infrastructure

Secret은 전용 Secret Manager를 사용한다.

46.1 Runtime

Workload가 필요한 Secret만 단기적으로 획득한다.

46.2 Container Environment Variable

장기 Credential 전체를 환경변수로 무분별하게 주입하지 않는 것을 권장한다.

46.3 Rotation

Deployment 재빌드 없이 Secret Rotation이 가능해야 한다.

47. Configuration Architecture

Configuration은 다음 계층으로 관리한다.

Platform Default

→ Environment

→ Tenant

→ Namespace

→ Resource

47.1 Config Store

Central Configuration Service 또는 선언적 Configuration Store를 사용할 수 있다.

47.2 Secret 분리

Configuration과 Secret을 동일 저장소에서 평문으로 관리하지 않는다.

48. Feature Flag

Feature Flag를 활용해 기능을 단계적으로 Release할 수 있다.

48.1 Scope

- Platform
- Environment
- Tenant
- Percentage

48.2 보안

Security Baseline을 비활성화하는 용도로 일반 Feature Flag를 사용하지 않는다.

49. Deployment Artifact

Deployment Artifact는 다음을 포함할 수 있다.

- Container Image
- Manifest
- Helm/Deployment Template
- Configuration Schema
- Migration
- SBOM
- Signature

49.1 Version

모든 Artifact는 추적 가능한 Version을 가져야 한다.

50. Infrastructure as Code

Infrastructure는 가능한 한 Code로 정의한다.

50.1 관리 대상

- Network
- Cluster
- Database
- Queue
- Storage
- IAM
- Monitoring
- DNS
- Secret Integration

50.2 변경

Pull Request, Review 및 CI 검증을 거쳐 적용하는 방식을 권장한다.

50.3 Drift

실제 Infrastructure와 IaC 정의 간 Drift를 탐지해야 한다.

51. Deployment Manifest

각 Service는 최소 다음 Deployment 정보를 가진다.

- Service Name
- Image
- Version
- Resource Request
- Resource Limit
- Replicas
- Health Check
- Environment
- Secret References

- Network Policy
 - Autoscaling
 - Deployment Strategy
-

52. Resource Request와 Limit

모든 Production Workload는 가능한 경우 Resource Request와 Limit을 설정한다.

52.1 대상

- CPU
- Memory
- GPU
- Ephemeral Storage

52.2 이유

- Scheduler 정확성
 - Noisy Neighbor 방지
 - 비용 측정
 - OOM 위험 통제
-

53. Horizontal Scaling

Stateless Service와 Worker는 Horizontal Scaling을 기본으로 한다.

53.1 Scale Signal

- CPU
- Memory
- Queue Depth
- Request Rate
- Latency

- Concurrent Execution

53.2 Worker

Worker는 CPU보다 Queue Depth나 Pending Task가 더 유용한 지표일 수 있다.

54. Vertical Scaling

Stateful Service와 일부 AI Model은 Vertical Scaling이 필요할 수 있다.

54.1 적용 대상

- Database
- Large Model Runtime
- Search Node

54.2 한계

Vertical Scaling만을 전체 성장 전략으로 사용하지 않는다.

55. Autoscaling Architecture

Autoscaling은 다음 수준에서 적용할 수 있다.

- API
- Worker
- Model Runtime
- Knowledge Worker
- Plugin Runtime

55.1 Scale Up

부하 증가에 빠르게 대응한다.

55.2 Scale Down

진행 중 Task가 있는 Worker를 강제 종료하지 않는다.

Graceful Drain을 수행한다.

56. Graceful Shutdown

Service 종료 시 다음을 수행한다.

1. Readiness Off
2. 신규 요청 차단
3. 진행 요청 완료
4. Worker Lease 해제
5. Checkpoint
6. Connection 종료
7. Process 종료

56.1 Timeout

Graceful Shutdown에도 최대 시간을 설정한다.

57. Worker Drain

Worker Deployment 또는 Scale Down 시:

Mark Draining

- Stop New Task
- Finish / Checkpoint Current Task
- Release Lease
- Shutdown

57.1 장기 Task

완료가 어려우면 Checkpoint 후 다른 Worker로 복구할 수 있어야 한다.

58. Health Check

각 Service는 최소 다음 Health 정보를 제공한다.

- Liveness
- Readiness

필요하면:

- Dependency Health
- Degraded Status

58.1 Liveness

프로세스가 복구 불가능한 상태인지 판단한다.

58.2 Readiness

현재 신규 Traffic을 처리 가능한지 판단한다.

58.3 외부 Dependency

외부 Model Provider 하나가 장애라고 API Service 자체를 무조건 Unready 처리하지 않는다.

59. Deployment Strategy

AGEX는 다음 전략을 지원할 수 있다.

- Rolling
- Blue-Green
- Canary

59.1 기본

일반 Service는 Rolling Update를 기본으로 사용할 수 있다.

59.2 고위험 변경

Core Runtime 또는 Model Gateway의 주요 변경은 Canary 또는 Blue-Green을 고려한다.

60. Rolling Deployment

새 Version을 순차적으로 배포한다.

60.1 조건

이전 Version과 신규 Version이 일정 시간 공존할 수 있어야 한다.

따라서 API 및 Database Compatibility가 필요하다.

60.2 실패

Health 또는 Error Rate가 기준을 초과하면 Rollout을 중단한다.

61. Canary Deployment

일부 Traffic 또는 Tenant에 신규 Version을 먼저 적용한다.

61.1 관측 대상

- Error Rate
- Latency
- Execution Failure
- Cost Change
- Security Event

61.2 Tenant Canary

특정 Test Tenant를 Canary 대상으로 사용할 수 있다.

62. Blue-Green Deployment

기존 환경과 신규 환경을 동시에 유지하고 Routing을 전환한다.

62.1 적용

- API Major Upgrade
- Runtime Major Upgrade
- Infrastructure Major Change

62.2 비용

두 환경을 동시에 유지해야 하므로 비용을 고려한다.

63. Rollback Architecture

Rollback은 Application Image만 이전 Version으로 변경하는 작업이 아니다.

검토 대상:

- Database Schema
- Configuration
- Event Schema
- Resource Schema
- Package Compatibility

63.1 자동 Rollback

다음 기준으로 자동 Rollback을 적용할 수 있다.

- Health Failure
- Error Rate
- Startup Failure

63.2 Data Migration

비가역적 Migration 이후에는 단순 Image Rollback을 하지 않는다.

64. Release Version

AGEX Platform Release는 다음 Component Version을 추적한다.

- Platform Version
- API Version
- Core Version
- Runtime Version
- Worker Version
- Schema Version

- Migration Version

64.1 Release Manifest

배포된 전체 Component Version 조합을 Release Manifest로 기록한다.

65. Compatibility During Deployment

신규 Control Plane과 기존 Data Plane이 일정 기간 공존할 수 있어야 한다.

65.1 Compatibility Matrix

다음 관계를 관리한다.

Control Plane Version

↔ Runtime Version

↔ Worker Version

↔ API Version

↔ Schema Version

65.2 지원 범위

지원되지 않는 Worker Version은 신규 Task를 할당받지 않도록 한다.

66. Runtime Version Routing

Execution Snapshot에 Runtime Requirement를 저장할 수 있다.

Scheduler는 호환되는 Worker에만 Task를 할당한다.

66.1 장기 Workflow

Platform Upgrade 중에도 시작 당시 호환 Runtime에서 계속 실행할 수 있어야 한다.

67. Database Backup

Production Database는 정기 Backup을 수행한다.

67.1 유형

- Full Backup
- Incremental
- Point-in-Time Recovery

67.2 Encryption

Backup도 암호화한다.

67.3 테스트

Backup 존재 여부만 확인하지 않고 정기 Restore Test를 수행한다.

68. Object Storage Backup

Object Storage는 Versioning, Replication 또는 Backup Policy를 적용할 수 있다.

68.1 Tenant 삭제

Backup Retention과 삭제 정책이 충돌하지 않도록 관리한다.

69. Search 및 Vector Recovery

Search와 Vector Index는 Derived Data이므로 원본에서 재생성 가능해야 한다.

69.1 Recovery 우선순위

원본 Database와 Object Store Backup을 우선 보호한다.

69.2 Snapshot

대규모 Index의 Recovery 시간을 줄이기 위해 Snapshot을 사용할 수 있다.

70. Disaster Recovery Architecture

DR은 최소 다음 Component를 고려한다.

- Database
- Object Storage
- Control Plane

- Runtime Configuration
- Secret Metadata
- Event State
- Deployment Manifest

70.1 DR Region

필요한 Service Tier에서는 별도 Recovery Region을 운영할 수 있다.

71. RPO / RTO Profile

정확한 수치는 Service Level 문서에서 정의한다.

Architecture는 최소 다음 등급 구조를 지원할 수 있다.

- Standard
- Enhanced
- Critical

각 Profile은 다음을 정의한다.

- Backup Frequency
 - Replication
 - Recovery Time
 - Recovery Point
 - Dedicated Resources
-

72. Multi-Zone Architecture

Production 주요 Component는 하나의 Failure Zone에만 의존하지 않도록 한다.

72.1 대상

- API
- Control Plane

- Runtime Worker
- Database Replica
- Queue
- Cache

72.2 Scheduler

Pod 또는 Instance를 여러 Failure Zone에 분산할 수 있어야 한다.

73. Multi-Region Architecture

Multi-Region 환경은 다음 형태를 지원할 수 있다.

- Active-Passive
- Active-Active
- Region-Specific Tenant Placement

73.1 권장 초기 구조

Region-Specific Tenant Placement + DR Region을 우선 고려할 수 있다.

Active-Active는 데이터 일관성과 운영 복잡성이 매우 높으므로 필요성에 따라 적용한다.

74. Global Control Plane

Multi-Region 환경에서 Tenant Registry와 Deployment Metadata를 관리하는 Global Control Plane을 둘 수 있다.

74.1 최소 데이터

Global Control Plane에는 실제 Tenant Content를 최소화한다.

74.2 Region Routing

Tenant의 Primary Region을 기반으로 요청을 Routing한다.

75. Region Data Plane

각 Region은 다음을 독립적으로 가질 수 있다.

- Runtime
- Database/Data Partition
- Queue
- Object Storage
- Knowledge Index
- Memory Index
- Model Gateway

75.1 Region 장애

허용된 Region Policy 안에서만 다른 Region으로 Failover한다.

76. Data Residency Deployment

Tenant 데이터는 Tenant Region Policy를 준수해야 한다.

76.1 적용 대상

- Database
- Artifact
- Knowledge
- Memory
- Runtime
- Backup
- Model Processing

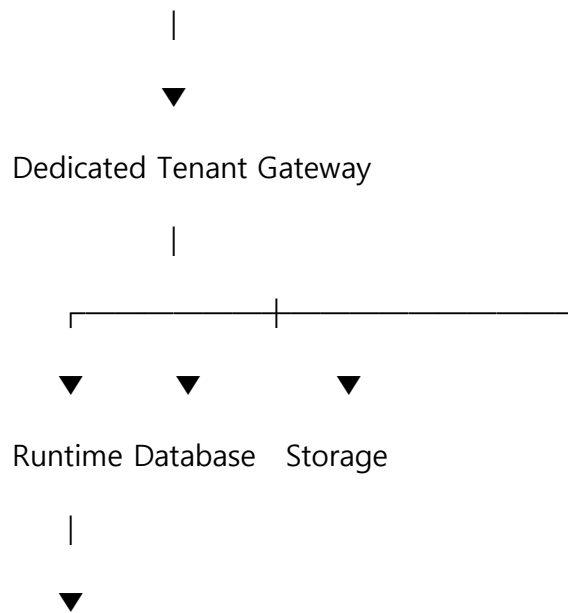
76.2 외부 Provider

외부 Model Provider 호출도 Data Residency Policy를 검토한다.

77. Dedicated Tenant Deployment

고보안 Tenant는 별도 Data Plane을 가질 수 있다.

Shared AGEX Control Plane



Private Model

77.1 공통 Contract

Dedicated Deployment도 동일 API 및 Resource Contract를 사용한다.

78. Private Cloud Deployment

Tenant의 Cloud Account 내부에 AGEX Data Plane 또는 전체 Platform을 배포할 수 있다.

78.1 필요한 구조

- Deployment Package
- IaC
- Runtime Image
- Secret Integration
- Monitoring Integration
- Upgrade Channel

78.2 중앙 Control

중앙 Platform과 연결될 경우 제한된 Management Channel을 사용한다.

79. On-Premises Deployment

On-Premises 환경은 Cloud Managed Service가 없을 수 있으므로 대체 가능한 Infrastructure Adapter를 지원해야 한다.

예:

- Self-Hosted Database
- Self-Hosted Queue
- Object Storage Compatible Service
- Local KMS/HSM
- Local Model Runtime

79.1 인터넷 없는 환경

Offline Package Distribution과 Update 방식을 지원할 수 있어야 한다.

80. Air-Gapped Deployment

완전 폐쇄 환경에서는 다음을 고려한다.

- Offline Container Registry
- Offline Package Repository
- Local Model
- Local Knowledge
- Offline License
- Offline Update Bundle

80.1 외부 Provider 기능

외부 API가 필요한 기능은 자동 사용되지 않는다.

81. Hybrid Deployment

Hybrid 환경에서는 Control Plane과 Data Plane 사이의 Contract를 명확히 정의해야 한다.

81.1 연결

- Private Link
- VPN
- Mutual TLS
- Outbound-Only Connector

81.2 Outbound-Only

방화벽 요구가 높은 환경에서는 Private Data Plane이 중앙 Control Plane으로 Outbound 연결하는 구조를 지원할 수 있다.

82. Runtime Connector

Remote Data Plane은 Runtime Connector를 통해 다음을 수행할 수 있다.

- Registration
- Health
- Capability Advertisement
- Task Polling
- Result Upload
- Configuration Sync

82.1 Security

Runtime Connector는 강한 Workload Identity와 mTLS를 적용할 수 있다.

83. Infrastructure Provider Abstraction

AGEX는 다음 Infrastructure 기능을 Interface로 추상화한다.

- Compute Provider
- Container Provider
- Storage Provider
- Database Provider
- Queue Provider
- Event Provider
- Cache Provider
- Search Provider
- Vector Provider
- Secret Provider
- KMS Provider

83.1 목적

Domain Logic이 특정 Cloud SDK에 직접 결합되지 않도록 한다.

84. Provider-Specific Optimization

Provider 독립성이 특정 Cloud 기능을 전혀 사용하지 않는다는 의미는 아니다.

Provider-specific 최적화는 Adapter 또는 Deployment Profile에서 사용할 수 있다.

Core Contract는 유지해야 한다.

85. Deployment Profile

AGEX는 환경별 Deployment Profile을 정의할 수 있다.

예:

Profile A — Local

- Single Node
- Local Database

- Mock Model

Profile B — Standard SaaS

- Shared Control Plane
- Shared Data Plane
- Multi-Zone

Profile C — Enterprise Dedicated

- Shared Control Plane
- Dedicated Runtime/Data

Profile D — Private

- Private Cloud / On-Premises
-

86. Capacity Planning

Capacity Planning은 최소 다음 지표를 사용한다.

- Tenant Count
- Active User
- API RPS
- Concurrent Executions
- Agent Iterations
- Workflow Tasks
- Model Tokens
- Plugin Calls
- Storage
- Knowledge Chunks
- Memory Count

86.1 AI 특성

단순 User 수보다 Model Call, Token, Agent Loop 및 Execution Concurrency가 Capacity에 더 큰 영향을 줄 수 있다.

87. Capacity Unit

AGEX는 내부 Capacity Unit을 정의할 수 있다.

예:

1 Execution Unit =

CPU

+ Memory

+ Expected Duration

+ Model Usage Class

+ Plugin Usage Class

실제 산식은 운영 단계에서 결정한다.

87.1 목적

- Capacity Forecast
- Tenant Quota
- Cost Planning
- Autoscaling

88. Performance Isolation

Control Plane의 대규모 관리 작업이 Runtime Latency에 영향을 주지 않도록 해야 한다.

예:

- Package Reindex
- Audit Export
- Knowledge Bulk Import

이 작업은 별도 Worker 또는 Queue를 사용한다.

89. Backpressure

부하가 처리 용량을 초과하면 무제한 Queue 적재로 대응하지 않는다.

89.1 적용

- Admission Control
- Rate Limit
- Queue Limit
- Tenant Fairness
- Retry-After
- Load Shedding

89.2 중요도

낮은 Priority 작업을 제한하고 Critical 작업에 Capacity를 보존할 수 있다.

90. Circuit Breaker Deployment

다음 외부 Dependency에 Circuit Breaker를 적용할 수 있다.

- Model Provider
- Plugin Provider
- External API
- Remote Storage

90.1 범위

Global Provider 장애와 Tenant Credential 장애를 구분해야 한다.

91. Deployment Observability

모든 Service와 Worker는 다음을 제공해야 한다.

- Metrics
- Structured Logs
- Distributed Trace
- Health
- Deployment Metadata

91.1 필수 Deployment Label

- Service
- Version
- Environment
- Region
- Cluster
- Instance/Pod
- Runtime Type

92. Metrics Architecture

주요 Infrastructure Metric:

- CPU
- Memory
- GPU
- Disk
- Network
- Pod Count
- Queue Depth
- DB Connection
- Cache Hit

- Error Rate
- Latency

주요 AGEX Metric:

- Execution Count
- Running Tasks
- Model Calls
- Plugin Calls
- Workflow Duration
- Agent Iterations

93. Logging Architecture

Log는 중앙 수집하되 다음을 보장해야 한다.

- Correlation ID
- Execution ID
- Tenant Scope where allowed
- Service
- Version
- Severity

93.1 Secret

Secret, Token 및 민감 Content는 Masking한다.

93.2 Raw AI Context

Model Prompt 전체를 기본 Infrastructure Log에 저장하지 않는다.

94. Distributed Tracing

API에서 시작하여 Runtime과 외부 호출까지 하나의 Trace로 연결한다.

예:

API

→ Core

→ Execution Manager

→ Agent Worker

→ Model Gateway

→ Model Provider

또는

Workflow

→ Plugin Gateway

→ Plugin Runtime

→ External Service

95. Deployment Dashboard

운영 Dashboard는 최소 다음을 제공한다.

- Cluster Health
- Service Health
- Deployment Version
- Queue Health
- Worker Capacity
- Model Gateway Health
- Database Health
- Error Rate
- Tenant Load
- Cost Signal

96. Alerting

Alert 조건 예:

- API Error Spike
- Queue Backlog
- Worker Failure
- Database Replication Lag
- Storage Capacity
- Model Provider Failure
- Plugin Failure
- GPU Exhaustion
- Security Incident

96.1 Alert Noise

일시적 단일 실패보다 지속성과 영향 범위를 고려해 Alert를 설계한다.

97. Deployment Security

Deployment Layer는 최소 다음을 적용한다.

- Image Signing
- Image Scan
- Non-Root
- Network Policy
- Secret Store
- Workload Identity
- RBAC
- Audit

- Encryption
 - Restricted Admin Access
-

98. Cluster Access Security

Production Cluster 직접 접근은 제한한다.

98.1 적용 가능 통제

- MFA
- JIT Access
- Bastion
- Short-Lived Credential
- Session Audit

98.2 CI/CD

일반 배포는 자동 Deployment Pipeline을 통해 수행한다.

99. Service Account

각 주요 Service는 독립 Service Account 또는 Workload Identity를 사용한다.

99.1 금지

모든 Pod가 동일 Cluster Admin Service Account를 사용하지 않는다.

99.2 최소 권한

필요한 Secret, API 및 Namespace만 접근하도록 한다.

100. CI/CD Architecture

권장 Pipeline:

Commit

→ Build

- Unit Test
 - Security Scan
 - Contract Test
 - Build Image
 - Generate SBOM
 - Sign Artifact
 - Integration Test
 - Stage Deploy
 - Validation
 - Approval
 - Production Deploy
-

101. GitOps

Deployment State를 Git 또는 동등한 Versioned Source에서 선언적으로 관리할 수 있다.

101.1 장점

- Change Review
- Audit
- Rollback
- Drift Detection
- Reproducibility

101.2 Secret

Secret 원문은 Git에 저장하지 않는다.

102. Release Approval

고위험 Production 배포는 다음을 요구할 수 있다.

- Test Pass
- Security Scan Pass
- Compatibility Pass
- Change Approval

102.1 긴급 Patch

Critical Security Patch는 별도 Emergency Release 절차를 사용할 수 있다.

103. Database Release Coordination

Application과 DB Migration을 하나의 원자적 Deploy로 가정하지 않는다.

다음 단계가 가능해야 한다.

Compatible Schema

→ App Deploy

→ Backfill

→ Verify

→ Remove Legacy

104. Feature Rollout

신규 기능은 Tenant 또는 Traffic 비율별로 단계적 활성화할 수 있다.

104.1 대상

- New Agent Runtime
- New Model Router
- New Workflow Engine
- New Retrieval Engine

104.2 Rollback

Feature Flag를 통해 신규 기능을 빠르게 비활성화할 수 있다.

105. Model Rollout

새로운 Model 또는 Provider는 일반 Application Release와 다르게 관리할 수 있다.

105.1 단계

- Register
- Evaluate
- Internal Test
- Limited Tenant
- General Availability

105.2 자동 교체 금지

새 Model이 등록되었다는 이유로 기존 Tenant Routing을 자동 변경하지 않는다.

106. Runtime Image Rollout

Runtime Image 변경은 실행 중 Workflow의 재현성에 영향을 줄 수 있다.

106.1 Version Pinning

Execution Snapshot이 필요한 Runtime Version을 기록한다.

106.2 Old Worker

장기 실행 지원을 위해 일정 기간 이전 Worker Version Pool을 유지할 수 있다.

107. Plugin Runtime Rollout

Plugin Framework Runtime 변경 시 Certified 및 Community Plugin을 각각 검증해야 한다.

107.1 Compatibility Test

Marketplace에 등록된 주요 Plugin과 자동 Compatibility Test를 수행할 수 있다.

108. Maintenance Mode

Platform 또는 Region에 Maintenance Mode를 적용할 수 있다.

108.1 동작

- 신규 실행 제한
- 일부 관리 API 제한
- 진행 중 실행 완료
- Maintenance 안내

108.2 긴급 Maintenance

Security 사고에서는 즉시 신규 실행을 차단할 수 있다.

109. Upgrade Architecture

Platform Upgrade는 다음 순서를 고려한다.

Compatibility Check

- Database Expand
- Control Plane Upgrade
- Runtime Manager Upgrade
- Worker Canary
- Worker Rollout
- Validation
- Cleanup Legacy

실제 순서는 변경 유형에 따라 조정한다.

110. Cluster Upgrade

Container Orchestration Platform 자체 Upgrade도 AGEX Release와 별도로 관리한다.

110.1 테스트

Staging에서 다음을 검증한다.

- Network
- Storage
- Workload Identity
- Autoscaling
- GPU
- Runtime Sandbox

111. Dependency Outage Handling

Infrastructure Dependency 장애 시 예상 동작을 정의한다.

111.1 Database

신규 State Mutation 중단.

111.2 Cache

Source of Truth로 Fallback 가능.

111.3 Queue

신규 Task Dispatch 지연.

111.4 Search

일부 Query 기능 Degraded.

111.5 Model Provider

Fallback Policy 적용.

112. Degraded Mode

Platform 전체를 즉시 실패시키지 않고 일부 기능을 제한해 운영할 수 있다.

예:

- Marketplace Search 장애 → 기존 설치 실행 가능

- Re-ranker 장애 → 기본 Retrieval 사용
- 특정 Model 장애 → 허용 Provider Fallback

112.1 보안 예외 금지

Degraded Mode에서도 Security Validation을 생략하지 않는다.

113. Failover

Failover 대상:

- Database
- API
- Queue
- Runtime Manager
- Region

113.1 자동 Failover

상태 무결성을 확인할 수 있는 Component에서만 자동 수행한다.

113.2 Split Brain

Stateful Component의 Split Brain을 방지해야 한다.

114. Failure Domain

Architecture는 다음 Failure Domain을 고려한다.

- Process
- Node
- Zone
- Cluster
- Region
- Provider

114.1 장애 설계

하나의 Failure Domain 손실이 어느 수준까지 영향을 주는지 명확히 정의해야 한다.

115. Chaos Testing

운영 준비 과정에서 다음 장애를 주입해볼 수 있다.

- Worker Kill
- Node Failure
- Network Latency
- Queue Delay
- Database Failover
- Model Provider Outage
- Region Dependency Failure

115.1 목적

Recovery와 Failure Isolation이 실제로 동작하는지 검증한다.

116. Backup Restore Testing

정기적으로 다음을 검증한다.

- Database Restore
- Object Restore
- Configuration Restore
- Tenant Restore
- Secret Metadata Recovery

Backup이 생성된 사실만으로 복구 가능하다고 판단하지 않는다.

117. Deployment Testing

Deployment Pipeline은 최소 다음을 검증해야 한다.

- Image Integrity
 - Manifest Validation
 - Config Validation
 - Secret Reference
 - Health Check
 - Migration
 - Service Connectivity
 - Network Policy
 - Rollback
-

118. Performance Testing

Deployment 환경에서 다음을 검증한다.

- API Throughput
 - Queue Throughput
 - Runtime Throughput
 - Model Gateway Throughput
 - Plugin Concurrency
 - Knowledge Retrieval Latency
 - Memory Retrieval Latency
-

119. Load Profile Testing

AI OS 특성을 고려해 다음 Profile을 테스트한다.

Profile A

다수의 짧은 Agent 실행.

Profile B

소수의 장시간 Workflow.

Profile C

대규모 Model Streaming.

Profile D

Knowledge Bulk Ingestion.

Profile E

Plugin Burst Traffic.

120. Cost-Aware Deployment

Deployment 구조는 Resource 사용량과 비용을 측정할 수 있어야 한다.

120.1 주요 비용

- Compute
- GPU
- Database
- Storage
- Network
- Search
- Queue
- Logging

120.2 Cost Attribution

Tenant 및 Workload 유형별 비용을 연결할 수 있어야 한다.

121. Resource Efficiency

다음 전략을 사용할 수 있다.

- Autoscaling
- Spot/Preemptible Worker for Safe Tasks
- Scale-to-Zero
- Batch Processing
- Storage Tiering
- Shared Model Runtime

121.1 안전성

비용 최적화가 실행 신뢰성과 Security Constraint를 훼손해서는 안 된다.

122. Spot / Preemptible Worker

중단 가능한 저위험 Task에 사용할 수 있다.

122.1 필수 조건

- Checkpoint
- Retry
- Idempotency
- No Irreversible Side Effect

122.2 고위험 Task

중단에 민감한 Action에는 사용하지 않을 수 있다.

123. Data Transfer Cost

Multi-Region 및 외부 Provider 호출은 Network Cost를 발생시킨다.

Routing 시 다음을 고려할 수 있다.

- Data Residency
- Latency
- Egress Cost

- Provider Cost

Security와 Residency가 Cost보다 우선한다.

124. Deployment Metadata

모든 실행 요청은 필요 시 현재 Deployment 정보를 추적할 수 있어야 한다.

예:

- Platform Version
- Runtime Version
- Worker Image
- Region
- Cluster

124.1 재현성

특정 Execution이 어떤 Runtime 환경에서 수행되었는지 확인하는 데 사용한다.

125. Environment Configuration Drift

Staging과 Production Configuration 차이를 관리한다.

125.1 허용 차이

- Endpoint
- Scale
- Credentials
- Region

125.2 금지 차이

핵심 Security 설정이 이유 없이 크게 달라지는 구조를 지양한다.

126. Deployment Governance

다음 변경은 관리 대상이다.

- Production Deployment
- Database Migration
- Runtime Major Update
- Network Policy Change
- Secret Provider Change
- Region Migration
- GPU Runtime Change

126.1 Change Record

중요 변경은 다음을 기록한다.

- Change ID
- Owner
- Version
- Risk
- Rollback Plan
- Approval
- Result

127. Deployment Runbook

각 주요 Component는 운영 Runbook을 가져야 한다.

예:

- API Failure
- Database Failure
- Queue Backlog
- Worker Exhaustion

- Model Gateway Failure
- Region Failure
- Rollback

상세 절차는 Operations Guide에서 정의한다.

128. Deployment Acceptance Criteria

Production Deployment Architecture는 최소한 다음을 충족해야 한다.

1. Control Plane과 Data Plane 책임이 구분된다.
2. Production Artifact가 Versioned되어 있다.
3. Container Image Integrity를 확인할 수 있다.
4. Secret이 Image에 포함되지 않는다.
5. Stateful Data가 외부 Store에 저장된다.
6. Runtime Worker를 Horizontal Scale할 수 있다.
7. Worker Graceful Drain이 가능하다.
8. Multi-Zone 배치를 지원할 수 있다.
9. Database Backup이 존재한다.
10. Restore Test가 가능하다.
11. Queue가 At-Least-Once를 전제로 설계된다.
12. Execution State가 Queue에만 존재하지 않는다.
13. Network Policy가 적용된다.
14. Workload Identity를 사용할 수 있다.
15. Deployment Rollback 절차가 존재한다.
16. Database Migration 전략이 존재한다.
17. Metrics, Logs, Trace를 수집할 수 있다.
18. Tenant Isolation이 Deployment Layer에서도 유지된다.

19. Region Policy를 적용할 수 있다.

20. Shared와 Dedicated Deployment가 동일 AGEX Contract를 사용한다.

129. Deployment Architecture 금지사항

다음 구조는 허용하지 않는다.

- Production 단일 서버 의존
- 실행 상태를 Worker Memory에만 저장
- Container Image 내부에 Secret 저장
- Production에서 latest Image만 사용
- 모든 Component가 동일 관리자 Credential 사용
- Database를 Public Internet에 직접 노출
- 모든 Cluster Service 간 All-to-All Network 허용을 기본값으로 사용
- Plugin Runtime과 Core를 동일 권한 Process에서 실행
- Queue를 Execution Source of Truth로 사용
- Worker 강제 종료 시 Checkpoint 없는 장기 실행 손실
- Deployment마다 Source를 다시 Build하여 환경별 Artifact가 달라지는 구조
- DB Breaking Migration과 Application Deployment를 무검증 동시 수행
- Region Policy를 무시한 자동 Failover
- Tenant별 비용을 구분할 수 없는 Shared Runtime
- Health Check 없는 Production Service
- Rollback Plan 없는 Major Release
- Backup Restore Test 없는 운영
- 검증되지 않은 Container Image 실행
- Production Cluster의 수동 변경을 기본 운영 방식으로 사용
- Cloud Provider SDK를 Domain Logic에 직접 사용

- Local Development 구조를 그대로 Production Architecture로 간주

130. Deployment Architecture 검증 기준

새로운 AGEX Component는 Deployment 전에 다음 질문을 통과해야 한다.

1. Control Plane인가 Data Plane인가.
 2. Stateless하게 만들 수 있는가.
 3. 어떤 State Store가 필요한가.
 4. 독립 Scaling이 필요한가.
 5. 어떤 Queue를 사용하는가.
 6. Tenant Context가 유지되는가.
 7. 어떤 Network 접근이 필요한가.
 8. 어떤 Secret이 필요한가.
 9. 어떤 Resource Limit이 필요한가.
 10. GPU가 필요한가.
 11. Health Check는 무엇인가.
 12. Graceful Shutdown이 가능한가.
 13. Checkpoint 또는 Retry가 필요한가.
 14. Failure가 다른 Service로 확산되는가.
 15. Multi-Zone 배포가 가능한가.
 16. Region Policy를 준수하는가.
 17. Rollback이 가능한가.
 18. Backup 또는 Rebuild 전략이 있는가.
 19. Observability가 가능한가.
 20. Provider를 변경해도 Domain Contract를 유지하는가.
-

131. Deployment Architecture의 최종 정의

AGEX Deployment Architecture는 AI Operating System의 Control Plane, Data Plane, Runtime, Model, Plugin 및 Data Component를 독립적으로 배포·확장·격리·복구할 수 있도록 구성하는 Cloud-Native 실행 인프라 구조이다.

AGEX는 하나의 Application Server에 모든 기능을 결합하지 않는다.

Resource 및 Policy를 관리하는 Control Plane과 실제 Agent·Workflow·Plugin·Model 실행을 처리하는 Data Plane을 논리적으로 분리하여 각각의 보안, 확장성 및 장애 특성에 맞게 운영한다.

Runtime Worker는 Stateless하게 유지하는 것을 원칙으로 하고 장기 실행 상태와 Checkpoint는 외부 Durable State에 저장한다.

Queue는 Task 전달 수단이며 실행 상태의 Source of Truth가 아니다.

Agent, Workflow, Plugin, Knowledge Processing 및 GPU Model Runtime은 부하와 보안 특성에 따라 서로 다른 Worker Pool로 분리할 수 있어야 한다.

AGEX는 Public SaaS뿐 아니라 Dedicated SaaS, Private Cloud, On-Premises, Hybrid 및 Air-Gapped 환경에서도 동일한 Resource와 API Contract를 유지하도록 설계한다.

Production Artifact는 Immutable, Versioned, Signed 방식으로 관리하며 Build된 동일 Artifact를 환경별로 Promotion하는 방식을 기본으로 한다.

Database, Object Storage, Queue, Search, Vector, Secret 및 Model Infrastructure는 Provider Adapter를 통해 추상화하여 특정 Cloud Vendor에 대한 Core 종속성을 최소화한다.

Deployment는 Rolling, Canary, Blue-Green 방식을 지원할 수 있어야 하며, Database Migration, Runtime Version 및 장기 Workflow의 Compatibility를 함께 관리해야 한다.

Multi-Zone, Backup, Restore, Disaster Recovery 및 필요 시 Multi-Region을 통해 장애 복구 능력을 확보한다.

모든 Deployment Unit은 Metrics, Logs, Trace, Health 및 Version Metadata를 제공해야 하며, Tenant 사용량과 비용도 Deployment Layer에서 추적 가능해야 한다.

AGEX Deployment Architecture는 다음 문장으로 최종 정의한다.

AGEX Deployment Architecture는 AI의 제어 영역과 실행 영역을 분리하고, 모든 서비스·Worker·데이터·모델을 독립적으로 확장·격리·복구할 수 있게 함으로써 AGEX를 다양한 인프라 환경에서 안정적으로 운영할 수 있도록 하는 AI Operating System의 물리적 실행

구조이다.

본 문서는 AGEX의 실제 Infrastructure, Container, Runtime, Data Plane, Scaling, Release, Backup 및 Disaster Recovery 구조를 규정하는 공식 Deployment Architecture 문서로 사용한다.